A stale comment, a silent failure

Why `z.init = 0.01` destroyed the time-block forecast study

Technical note

12 July 2026 (rev. 18 July 2026)

Abstract

The time-block forecast study (`code/analysis/time_block_forecast/simstudy/`) reported that the AR(2) model forecasts *four times worse* than an i.i.d.-in-time baseline, with the gap growing monotonically in lead time. That result is an artifact of a single argument, `z.init = 0.01`, passed to `mcmc()` on the strength of a comment that was true when it was written and false by the time it was read. This note explains why the comment was wrong, derives the cascade from that one number to a predictive standard deviation of 104 against a data marginal of 4.38, and documents the verification — including the effect on the study's own scores, where the fix halves the AR(2) CRPS and cuts its Brier by a third. It then records two findings from the cleanup: the model non-identifiability the bug exploited persists as a milder, stochastic "reflected" branch that corrupts *every* proper score (Brier included, not only the multivariate ones), and the same fatal default still lies dormant behind a guard in a sibling backend. The deeper point is not the bug: it is that the workaround converted a *loud* failure into a *silent* one.

1 The comment

`run-settings.R` carried this justification for overriding the default initial value of the latent skew process z:

```
# z.init = 0 (the backend default) is fatal for the temporal/AR(2) z
# update: hn.cop(0, sig) = qnorm(phn(0, sig)) = -Inf, the MH ratio R
# becomes NaN, and the `if (!is.na(R))` guard in updateZTS_AR2
# silently rejects every proposal -- z and phi.z stay stuck at 0,
# producing all-NaN forecasts. A small positive initial value lets
# hn.cop evaluate finitely on the very first iteration.
fit <- mcmc(..., z.init = 0.01, ...)
```

Every sentence about the *mechanism* is correct. The sentence about the *default* is not.

2 What the backend actually does

The model couples z to a latent Gaussian process through a half-normal copula. With $\sigma = \tau^{-1/2}$,

$$z^\star \;=\; \mathrm{hn.cop}(z,\sigma) \;=\; \Phi^{-1}\big(\,\mathrm{phn}(z,\sigma)\big) \;=\; \Phi^{-1}\big(2\,\Phi(z/\sigma)-1\big). \tag{1}$$

At $z = 0$ this gives $\Phi^{-1}(0) = -\infty$, and the Metropolis ratio degenerates to `NaN`. That failure was real, and it was *fixed in the backend*. From `code/R/ar2/mcmc_ar2.R:249--250`:

```
if (is.null(z.init)) {
    z.init <- 0.6745 / sqrt(tau.init)  # median of HN(1/sqrt(tau.init)) -> z.
        star = 0
}
```

The signature default is `z.init = NULL`, *not* 0. When `NULL`, z is initialised at the **median of its own half-normal marginal**. This choice is exact: since $0.6745 = \Phi^{-1}(0.75)$, substituting $z = 0.6745\,\sigma$ into (1) gives

$$z^\star \;=\; \Phi^{-1}\big(2\Phi(0.6745) - 1\big) \;=\; \Phi^{-1}(2 \times 0.75 - 1) \;=\; \Phi^{-1}(0.5) \;=\; 0, \tag{2}$$

dead centre of the copula's Gaussian scale, for any τ. The backend's own `README.md` documents this fix, its symptom, and its rationale — and closes with the sentence that seals the trap:

> *"Callers who pass* ***`z.init`*** *explicitly are unaffected."*

3 Why the comment was true once, and false later

The chronology is the whole story.

1. The backend default *was* $z.init = 0$. It produced $-\infty$, then `NaN`, then a frozen chain. This is the bug the comment describes.
2. `run-settings.R` worked around it locally by passing `z.init = 0.01`. The $-\infty$ disappeared; the study ran.
3. The backend was then fixed *properly*, changing the default to `NULL` $\mapsto$ the half-normal median, and documenting it.
4. The correct fix could not reach `run-settings.R`, precisely because it passed `z.init` explicitly — exactly the case the README excludes.
5. Nobody removed the now-obsolete workaround, and its comment — an accurate description of a world that no longer existed — kept the workaround looking justified.

The comment did not merely go stale. It *actively defended* the line that had to be deleted.

4 What 0.01 actually does

Take `tau.init = 1`, so $\sigma = 1$. Equation (1) gives

`z.init`	$z^\star = \mathrm{hn.cop}(z, 1)$	
0	$-\infty$	the original bug: `NaN`, chain frozen
0.01 *(the workaround)*	$\mathbf{-2.41}$	finite, but a 2.4σ tail start
0.6745 *(the correct default)*	$\mathbf{0}$	the centre of the copula scale

The workaround did not restore the chain to a sensible starting point. It moved it from an *impossible* one to an *extremely improbable* one. A random-walk Metropolis chain started 2.4 standard deviations into the tail of the very space it must explore does not, in practice, come back.

5 The cascade

5.1 Step 1: z freezes

Started at $z^\star = -2.41$, the chain never reaches the bulk. Across 20,000 iterations the fitted z stays at ≈ 0.2, while its own model distribution, $z \sim \mathrm{HN}(0, \sigma)$ with $\sigma \approx 2$, demands

$$\mathbb{E}[z] \;=\; \sigma\sqrt{2/\pi} \;\approx\; 1.6. \tag{3}$$

The fitted z is a factor of 7–8 below the value the model itself prescribes. Its maximum over the entire retained chain is 0.30; in a healthy fit it reaches 15.

5.2 Step 2: the AR coefficient chases the freeze

A frozen z is, as a time series, nearly constant — and a nearly constant series looks maximally persistent. The AR(2) update on $z^\star$ obliges, driving $\hat{\phi}_{1z} \to 1.11$ (truth: 0.80), i.e. toward a unit root. The near-unit-root prior then pins z *harder*. The freeze is self-reinforcing: $\phi \mid z \to$ unit root if z looks smooth; $z \mid \phi \to$ smooth if ϕ is a unit root.

5.3 Step 3: λ and β_0 compensate

The data see the skew channel only through the *product* λz, in

$$\mu \;=\; \beta_0 \;+\; \lambda\,\bar{z}. \tag{4}$$

This is a ridge: $(\beta_0, \lambda, \bar{z})$ can slide along it freely provided μ is preserved. With $\bar{z}$ pinned 7× too small, the sampler restores μ by sending λ and β_0 to absurd values. All three blocks reproduce the data mean of 14.5 *exactly*:

	β_0	λ	$\bar{z}$	$\mu = \beta_0 + \lambda\bar{z}$
truth	10	+3	≈ 1.6	≈ 14.5
block 1 (healthy)	10.36	+2.91	1.42	**14.50**
block 3 (*broken*)	36.41	**−97.75**	0.22	**14.92**

This is why the failure is silent. The likelihood is satisfied. The fitted mean is right. Nothing inside the fit looks wrong. Only the individual parameters are catastrophic — and λ has even flipped sign.

5.4 Step 4: the forecast exposes the inconsistency

`forecast_block` is correct, and that is exactly why it detonates. It does not reuse the frozen z; it redraws z from the model's own law, $z_{\mathrm{fc}} \sim \mathrm{HN}(0, \tau_{\mathrm{fc}}^{-1/2})$, which by (3) has mean ≈ 1.6. It then multiplies by the corrupted λ:

$$\lambda\, z_{\mathrm{fc}} \;\approx\; (-97.75)(1.6) \;\approx\; -156, \qquad \mathrm{sd}\left(\lambda z_{\mathrm{fc}}\right) \;=\; |\lambda|\,\sigma\sqrt{1 - 2/\pi} \;\approx\; 118. \tag{5}$$

Against a data marginal standard deviation of 4.38, the observed predictive SD was 104 and the predictive median was dragged far negative — for a quantity whose data never leave 14.5 ± 4.4.

The forecast is the *only* place in the pipeline where z is drawn from its model distribution rather than read from the stuck chain. It is therefore the only place the inconsistency can become visible. The forecaster was not the bug; it was the detector.

6 Verification

The chain was reproduced bit-for-bit (same seed, same sequential block order), then rerun with the single argument `z.init = 0.01` removed and nothing else changed.

		with `z.init = 0.01`			**default `z.init`**		
quantity (truth)		blk 1	blk 2	blk 3	blk 1	blk 2	blk 3
$\bar{z}$	(≈ 1.6)	1.42	0.20	0.22	0.77	1.46	**1.68**
λ	($+3$)	2.91	-55.8	-97.8	5.81	2.90	**2.80**
β_0	(10)	10.36	25.19	36.41	10.34	10.45	**10.36**
ϕ_{1z}	(0.80)	0.55	0.94	1.11	0.61	0.58	**0.62**
predictive SD, lead 15	(4.38)	5.79	**41.9**	**104.4**	9.89	5.04	**4.94**

Deleting one argument returns λ to $+2.80$, β_0 to 10.36, $\bar{z}$ to 1.68, and the lead-15 predictive SD to 4.94 against a marginal of 4.38 — i.e. the forecast now converges to climatology, as an AR forecast must.

Consequence, in the currency of the study. Every score in the previous time-block study was computed from this corrupted predictive sample. Rescoring the setting-4 study (five blocks $\times$ five datasets) before and after the one-argument fix makes the damage concrete in the scores themselves, not merely in the predictive SD:

	before (`z.init=0.01`)	**after** (fixed)	change
AR(2) CRPS (mean)	5.95	2.64	-56%
AR(2) Brier q_{95}	0.104	0.070	-33%
i.i.d. CRPS (mean)	2.33	2.41	$+3\%$
i.i.d. Brier q_{95}	0.053	0.059	$+11\%$
AR(2)$-$i.i.d. CRPS gap	$+3.62$	$+0.23$	
(clean 21 blocks)		-0.015 ($p = 0.78$)	
AR(2)$-$i.i.d. Brier gap	$+0.051$	$+0.011$	
(clean 21 blocks)		$+0.0001$ ($p = 0.97$)	

The i.i.d. method barely moves — it never runs the temporal z update the bug lives in — while the AR(2) method's CRPS more than halves and its Brier falls by a third. Once the four residual reflected blocks are excluded (§7), the two methods are a statistical tie on both scores. The headline finding, "AR(2) forecasts four times worse than i.i.d.," was never a statement about temporal pooling. It was a statement about an initial value.

7 The residual: the same ridge, entered a different way

The fix removes the catastrophic corner but not the ridge (4). The bug forced *every* chain onto the ridge deterministically, through the z coordinate; with that door shut, a minority of chains still drift onto it stochastically during burn-in, through the (λ, β_0) coordinates. On the guarded block-1 study these "reflected" fits appear in about a quarter of cells: λ flips to -4 to

-15, β_0 rises to 26–35 so that $\beta_0 + \lambda\bar{z}$ still reproduces the data mean, $\bar{z}$ stays correct (so the level-based assertion below *passes*), and the predictive SD inflates as $\max_h \mathrm{sd}(\hat{y}_h) \approx 1.5\,|\hat{\lambda}|$. This is a genuine non-identifiability of the model, milder than the bug (predictive SD 6–21, not 104), and it is documented separately in the companion note `tex/lambda_phiz_ridge`. Two practical consequences belong here, because both were discovered while cleaning up after this bug.

7.1 A reflected fit corrupts every proper score, not only the joint ones

It is tempting to hope that a latent-process pathology only shows up in multivariate scores (energy, variogram) that credit temporal dependence, so that a study reporting the univariate Brier score could ignore it. It cannot. Refitting the five strongest reflected cells bit-for-bit and scoring the reflected chain against its healthy replacement on all four scores:

cell	$\hat{\lambda}$ (refl. $\to$ heal.)	Brier q_{95}	CRPS	energy	variogram
s5 d3 m2	$-15.2 \to +3.3$	$0.109 \to 0.132$	$1.94 \to 2.15$	$9.9 \to 7.2$	$243 \to 93$
s4 d6 m2	$-8.9 \to +2.9$	$0.138 \to 0.135$	$5.59 \to 5.48$	$21.5 \to 18.3$	$430 \to 302$
s4 d5 m2	$-7.3 \to +3.9$	$\mathbf{0.021} \to \mathbf{0.002}$	$2.64 \to 1.57$	$14.3 \to 11.8$	$276 \to 179$
s4 d5 m1	$-7.2 \to +3.5$	$\mathbf{0.027} \to \mathbf{0.003}$	$3.19 \to 1.90$	$14.6 \to 12.2$	$301 \to 178$
s4 d10 m1	$-5.0 \to +3.0$	$0.056 \to 0.023$	$2.87 \to 2.21$	$11.6 \to 10.8$	$234 \to 144$
mean relative damage		(see text)	+32%	+21%	+78%

Every score is worse under reflection — the inflated $|\lambda|$ widens the *marginal* forecast at every lead, and no proper score rewards overdispersion. The ordering, though, is the opposite of the intuition above: the energy score is the *most* robust (+21%), the pure-dependence variogram the least (+78%), and CRPS sits between (+32%). The Brier score is damaged too, and most sharply exactly where it matters: on the three easy datasets (s4 d5, d10) whose healthy fit gets the tail nearly right, the reflected chain multiplies the Brier error several-fold ($0.002 \to 0.021$, $0.003 \to 0.027$, $0.023 \to 0.056$). On the two hard datasets (s5 d3, s4 d6), where even the healthy fit scores poorly, an overdispersed forecast happens to hedge, so the Brier barely moves or even improves — which is not a reprieve but a warning: a broken fit can win a single validation window by luck, so the score itself cannot be used to detect the pathology. *Focusing on the Brier score does not make the identification problem ignorable; it only makes it intermittent and harder to see.*

7.2 The same default still lurks in a sibling backend

The bug's root cause — a signature default that maps to $\mathrm{hn.cop}(0, \sigma) = -\infty$ — was fixed in `ar2` but survives as `z.init = 0` in the `prop` backend (`code/R/prop/mcmc_prop.R`). It is currently harmless: `prop` refuses temporal-z blocks with a `stop()` guard, and its non-temporal Gibbs update overwrites the initial value on the first sweep. But it is a dormant landmine. Enabling temporal z there — the obvious first step for a future extension — would re-arm the original freeze exactly; a direct test (same shared `updateZTS_AR2` machinery, `z.init = 0`) reproduces it on contact: z frozen, ϕ_z variance 0, `NaN` warnings by the hundred. We have synced the default to `NULL` $\mapsto$ the half-normal median, so the trap is disarmed before anyone steps on it.

8 The real lesson

The original bug was **loud**: $-\infty$, then `NaN`, then ϕ_z identically zero for every iteration. It announced itself. Anyone looking at the output would have stopped.

The workaround silenced it. With `z.init = 0.01` there is no $-\infty$, no `NaN`, no all-zero coefficient. There is a chain that mixes, a posterior that concentrates, a fitted mean that matches the data, and a set of scores that look entirely publishable. The failure did not go away; it went *underground*, and it stayed there for a full study.

> A workaround that removes the *symptom* without removing the *cause* does not make a system safer. It makes the next failure quieter.

What should have caught it. Two assertions, each one line, each checkable without any knowledge of this bug:

1. **Self-consistency of the fit.** The model asserts $z \sim \mathrm{HN}(0, \tau^{-1/2})$, hence $\mathbb{E}[z] = \tau^{-1/2}\sqrt{2/\pi}$. The fitted $\bar{z}$ was $7\times$ smaller. *A fit that violates its own generative assumption by a factor of seven is not a fit.*
2. **Recovery of known truth.** This is a *simulation* study: the truth is $\lambda = +3$. The fit returned $\lambda = -98$. The sign alone is disqualifying. Simulation studies have ground truth available for free and should assert against it automatically.

Neither assertion requires suspecting the initialisation. Either would have failed on the first block, in minutes, instead of after forty hours and a false conclusion.